



UNIVERSITÀ DI PARMA

DIPARTIMENTO DI SCIENZE MATEMATICHE, FISICHE E INFORMATICHE

Corso di Laurea Triennale in Informatica

Progettazione e Sviluppo di un Gestionale Interno per l'Ottimizzazione dei Processi Aziendali

CANDIDATO:
Martin Trajkovski

RELATORE:
Prof. Vincenzo Arceri

MATRICOLA:
335566

Talk is cheap. Show me the code.

Alla mia famiglia, per il loro amore, supporto e instancabile fiducia.

Indice

1	Introduzione	3
1.1	Dieffetech	3
1.2	Il gestionale	4
1.3	Architettura di un'applicazione	5
2	Il gestionale	7
2.1	Cause della Ristrutturazione	7
2.2	Confronto tra il Vecchio e il Nuovo Gestionale	8
2.2.1	Scelte Tecnologiche	8
2.2.2	Interfaccia utente	8
2.2.3	Prestazioni	8
2.2.4	Sicurezza	9
2.3	React	9
2.3.1	Architettura di un Progetto	10
2.4	Laravel	11
2.4.1	Sail	12
2.4.2	Sanctum	12
2.4.3	Telescope	13
2.5	Generazione del CRUD	13
2.6	API	16
2.6.1	Le rotte API nel progetto	16
2.6.2	Funzionamento delle rotte	17
3	Contributo Personale	19
3.1	Prima e ora	20
3.2	Tabella dei Clienti e Leads	21
3.2.1	Configurazione della CrudDataTable	21
3.3	Appuntamenti	24
3.4	Task	27
3.5	Offerte	29
3.5.1	Offer Model	31
3.5.2	Children e Parents	34

INDICE

3.6	Morph Relations	35
3.6.1	Referenti	36
3.6.2	Media	38
3.7	Scope Filters	39
3.8	Offerta Accettata	40
4	Possibili miglioramenti	43
4.1	Lavori in corso	43
4.2	AI	43
4.3	Aggiornamenti e aggiunte	44
5	Conclusione	45

Capitolo 1

Introduzione

Durante il mio percorso presso Dieffetech, ho avuto l'opportunità di contribuire allo sviluppo e al miglioramento del gestionale interno dell'azienda.

Questo sistema rappresenta uno strumento cruciale per l'ottimizzazione dei processi aziendali e la gestione delle attività quotidiane, favorendo l'efficienza operativa e migliorando l'organizzazione interna.

1.1 Dieffetech

Dieffetech S.r.l., con sede a Codogno (LO) in Via Sergio Ramelli, 24, è una dinamica software house specializzata nella creazione di soluzioni digitali personalizzate. Offre servizi di sviluppo software, siti web, applicazioni mobili e piattaforme e-commerce, mirando a soddisfare le esigenze specifiche dei clienti.

L'azienda si distingue per l'approccio consulenziale, collaborando strettamente con i clienti per sviluppare progetti e strumenti che favoriscano il successo aziendale. Tra i suoi clienti figurano realtà di rilievo come The European House – Ambrosetti, EdiliziAcrobatica, Iperal, Zucchetti e TIM Academy.

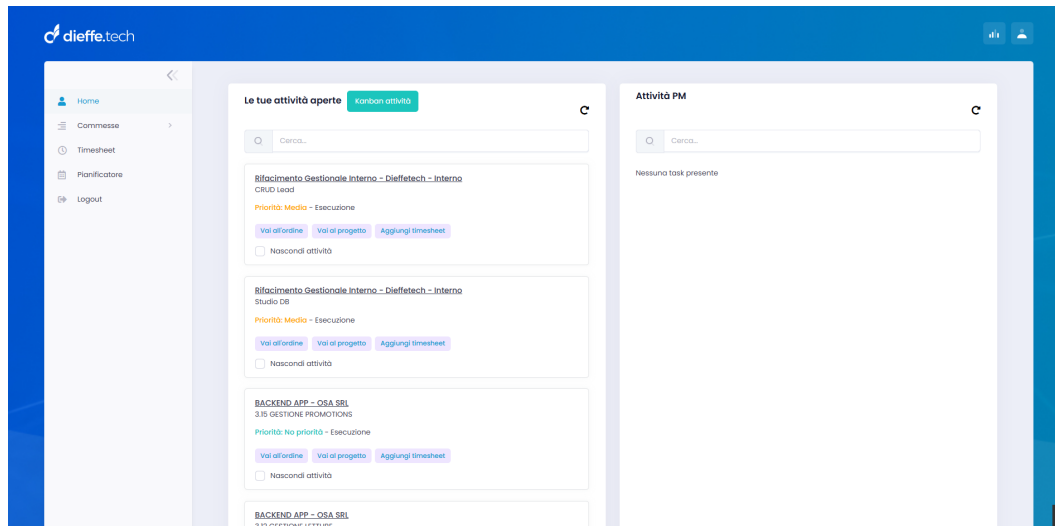
Dieffetech si impegna a fornire assistenza continua, supporto tempestivo e soluzioni innovative, garantendo prodotti testati, esteticamente gradevoli e funzionali. La sua missione è contribuire al miglioramento dei processi aziendali attraverso l'adozione di tecnologie avanzate e soluzioni digitali su misura.



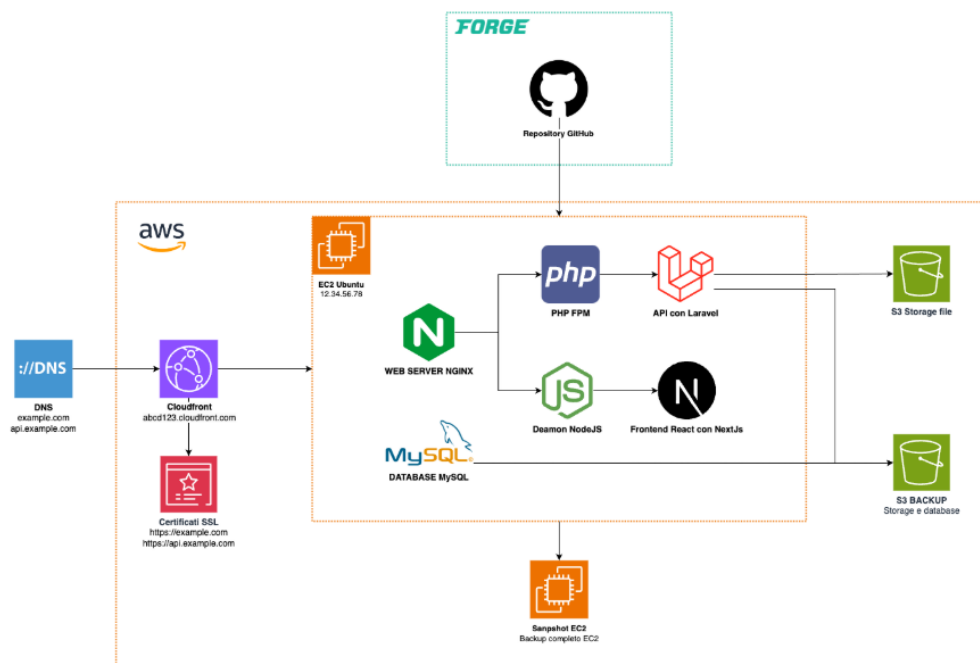
1.2 Il gestionale

Il gestionale interno sviluppato da Dieffetech è una piattaforma software progettata per automatizzare e ottimizzare i principali processi aziendali. Grazie alla gestione centralizzata delle informazioni e delle attività, il sistema facilita i dipendenti nello svolgimento delle loro mansioni quotidiane.

Le sue funzionalità principali includono l'assegnazione di task giornalieri, la visualizzazione del calendario delle presenze, la gestione avanzata dei clienti, il monitoraggio dei progetti avviati e il controllo dettagliato dei costi aziendali. Queste caratteristiche rendono il gestionale uno strumento fondamentale per aumentare l'efficienza operativa, riducendo gli errori manuali e semplificando la gestione dei flussi di lavoro.



1.3 Architettura di un'applicazione



L'architettura di un'applicazione in Dieffetech è ospitata su AWS, progettata per garantire alte prestazioni, sicurezza e scalabilità. I principali componenti e servizi utilizzati sono:

- **Gestione dei DNS:** I domini `example.com` e `api.example.com` sono gestiti tramite provider DNS come Register o Aruba. I record CNAME sono configurati per puntare a AWS CloudFront, che funge da firewall per proteggere l'applicazione da attacchi DDoS, SQL Injection e Bot. Per poter usare i domini assieme a AWS CloudFlare è indispensabile l'uso di certificati SSL (`https`) di AWS Certificate Manager.
- **Server:** L'applicazione è ospitata su un'istanza EC2 con sistema operativo Ubuntu. La sicurezza dell'istanza è garantita da un Security Group che limita l'accesso alle porte:
 - HTTP (porta 80)
 - HTTPS (porta 443)
 - SSH (porta 22), limitata a IP specifici
 - MySQL (porta 3306), limitata a IP specifici

- **Web Server:** Il server web utilizzato è Nginx, che gestisce le richieste provenienti da AWS CloudFront. Le richieste vengono poi inoltrate al frontend basato su Next.js e alle API implementate con Laravel.
- **Database e Storage:** Il database principale è MySQL, scelto per la sua robustezza e affidabilità. Per lo storage, viene utilizzato il disco locale dell'istanza EC2 per dati temporanei e AWS S3 per dati durevoli e scalabili.
- **Backup:** Sono adottate due strategie di backup per garantire la sicurezza dei dati:
 - Snapshot EC2: Backup giornalieri dell'istanza EC2 per garantire un rapido ripristino in caso di guasto.
 - Backup S3: Backup regolari dei dati e del database su AWS S3 per proteggere i dati a lungo termine.
- **Deploy:** Il codice sorgente è gestito tramite GitHub e distribuito automaticamente su EC2 tramite Forge. Questo flusso di lavoro automatizzato riduce i tempi di deploy e gli errori manuali, assicurando una distribuzione rapida e sicura delle nuove versioni dell'applicazione.

Capitolo 2

Il gestionale

Il gestionale interno di Dieffetech è stato rifatto per rispondere alle esigenze di efficienza, scalabilità e semplicità di gestione che l'azienda aveva maturato nel corso degli anni.

Questo capitolo descrive le motivazioni dietro il rifacimento del sistema e le tecnologie adottate, concentrandosi sulle principali differenze tra il vecchio e il nuovo gestionale, con un focus su Laravel e React, nonché sugli strumenti utilizzati per il suo sviluppo.

2.1 Cause della Ristrutturazione

Il rifacimento del gestionale interno di Dieffetech è stato motivato dalla necessità di modernizzare un sistema che, sebbene funzionante, aveva raggiunto dei limiti a causa delle tecnologie obsolete utilizzate.

In precedenza, il gestionale si basava su Yii per il backend, con l'utilizzo di JavaScript, HTML e CSS per il frontend. Sebbene queste tecnologie fossero adatte per le esigenze iniziali dell'azienda, non garantivano più la scalabilità e le prestazioni necessarie per gestire un flusso di lavoro crescente e un numero sempre maggiore di dati da elaborare.

La decisione di adottare Laravel per il backend e React per il frontend è stata determinata dalla necessità di rendere il sistema più robusto, flessibile e facilmente manutenibile.

Laravel, con la sua architettura solida e il suo ecosistema di strumenti, ha consentito di creare un backend più sicuro ed efficiente. React, d'altra parte, ha permesso di migliorare l'interfaccia utente, rendendola dinamica e reattiva, con tempi di risposta molto più rapidi rispetto alla versione precedente. Questa combinazione di tecnologie ha portato a una gestione centralizzata delle informazioni e a una migliore esperienza utente per i dipendenti di Dieffetech.

2.2 Confronto tra il Vecchio e il Nuovo Gestionale

Il rifacimento del gestionale ha comportato numerosi cambiamenti rispetto alla versione precedente. Le principali differenze riguardano sia la tecnologia utilizzata che l'esperienza dell'utente.

Qui di seguito sono descritte le differenze principali tra il vecchio e il nuovo sistema:

2.2.1 Scelte Tecnologiche

- **Vecchio sistema:** Il precedente sistema si avvaleva del framework `Yii` per il backend, mentre per il frontend si utilizzavano tecnologie come `JavaScript`, `HTML` e `CSS`. Sebbene queste tecnologie fossero adeguate per l'inizio, si rivelavano meno scalabili e modulari man mano che cresceva la complessità del sistema.
- **Nuovo sistema:** Il nuovo gestionale utilizza `Laravel` per il backend e `React` per il frontend. `Laravel` offre un'architettura robusta per la gestione delle operazioni lato server, mentre `React` consente un'interfaccia utente più dinamica e reattiva, migliorando l'esperienza complessiva dell'utente.

2.2.2 Interfaccia utente

- **Vecchio sistema:** L'interfaccia utente risultava funzionale, ma limitata in termini di flessibilità. Le interazioni non erano molto fluide e il sistema non era progettato per adattarsi facilmente alle evoluzioni delle necessità aziendali.
- **Nuovo sistema:** L'interfaccia utente è stata riprogettata per essere più moderna, interattiva e user-friendly. L'adozione di `React`, ha reso le transizioni tra le pagine significativamente più rapide, migliorando l'esperienza dell'utente finale e riducendo il rischio di errori.

2.2.3 Prestazioni

- **Vecchio sistema:** Le prestazioni del vecchio sistema erano sufficienti ma tendenti a rallentare quando il volume di dati e richieste cresceva, soprattutto nelle operazioni di gestione dei dati.

- **Nuovo sistema:** "Il nuovo sistema garantisce un notevole miglioramento delle prestazioni, grazie a un'architettura più efficiente. **React** migliora la velocità di rendering delle pagine, mentre **Laravel** ottimizza la gestione delle richieste lato server, riducendo i tempi di risposta e aumentando la reattività.

2.2.4 Sicurezza

- **Vecchio sistema:** Il sistema precedente non offriva gli stessi livelli di protezione e gestione sicura degli accessi che sono oggi richiesti per proteggere i dati sensibili.
- **Nuovo sistema:** Grazie all'integrazione di **Laravel Sanctum**, il nuovo gestionale offre un sistema di autenticazione sicuro basato su token CSRF, migliorando la protezione dei dati aziendali e garantendo un livello di sicurezza più elevato.

2.3 React

React è una libreria JavaScript sviluppata da Jordan Walke (un software engineer di Meta) nel 2013, progettata per creare interfacce utente (UI) reattive e dinamiche. Si distingue principalmente per la sua capacità di aggiornare in modo efficiente solo le porzioni di UI che cambiano, riducendo così il carico sulle risorse del sistema e migliorando l'esperienza utente. Utilizzando un approccio basato su componenti, React consente agli sviluppatori di suddividere le interfacce in parti modulari, ognuna con il proprio stato e comportamento, che possono essere riutilizzate e testate in modo indipendente.

Una delle caratteristiche distintive di React è il Virtual DOM (Document Object Model), un'astrazione che permette di migliorare le performance durante il rendering delle interfacce utente. Invece di aggiornare direttamente il DOM del browser, React prima aggiorna una rappresentazione in memoria (il Virtual DOM) e poi calcola la differenza con il DOM effettivo, applicando solo le modifiche necessarie. Questo approccio riduce significativamente i tempi di rendering, soprattutto per le applicazioni con una grande quantità di dati e componenti dinamici.

React segue il paradigma della programmazione dichiarativa, in cui gli sviluppatori descrivono come dovrebbe apparire la UI in ogni stato e React si occupa di aggiornare il DOM in modo efficiente quando lo stato cambia. Questo paradigma rende il codice più leggibile, facilmente manutenibile e scalabile, particolarmente utile in progetti di grandi dimensioni.

L'adozione di React nel rifacimento del gestionale di Dieffetech ha portato a una notevole miglioria nell'interazione dell'utente con il sistema. Grazie alla sua velocità di rendering e alla modularità dei componenti, l'interfaccia utente è diventata più fluida e reattiva, con una gestione dinamica dei dati in tempo reale. React ha permesso di creare un'esperienza più coinvolgente per gli utenti, riducendo al minimo i tempi di attesa e migliorando l'usabilità complessiva della piattaforma.

2.3.1 Architettura di un Progetto

Un tipico progetto React ha la seguente struttura delle cartelle:

```
react/
|-- node_modules/
|-- public/
|   |-- index.html
|   '-- favicon.ico
|-- src/
|   |-- components/
|   |-- forms/
|   |-- tables/
|   |-- styles/
|-- .gitignore
|-- .env
|-- package-lock.json
|-- package.json
'-- README.md
```

La cartella `src` è il cuore del progetto React, poiché contiene tutti i file sorgente necessari per il funzionamento dell'applicazione. Al suo interno, la cartella `components/` raccoglie tutti i componenti riutilizzabili, come il `CrudDataTable`, che gestisce le tabelle dinamiche per la visualizzazione e l'interazione con i dati. Inoltre, all'interno della cartella `src/`, esistono altre due cartelle fondamentali:

- **forms/**: Contiene tutti i moduli (form) necessari per la creazione e gestione di nuovi dati. Ogni tabella avrà il proprio form dedicato, consentendo una gestione centralizzata e modulare della creazione dei record.
- **tables/**: Contiene i componenti che gestiscono la visualizzazione delle tabelle. Questi componenti sono responsabili della gestione dei dati provenienti dal backend, e vengono visualizzati all'interno delle diverse tab

nel `baseMenu` del gestionale, permettendo una navigazione fluida tra le varie sezioni.

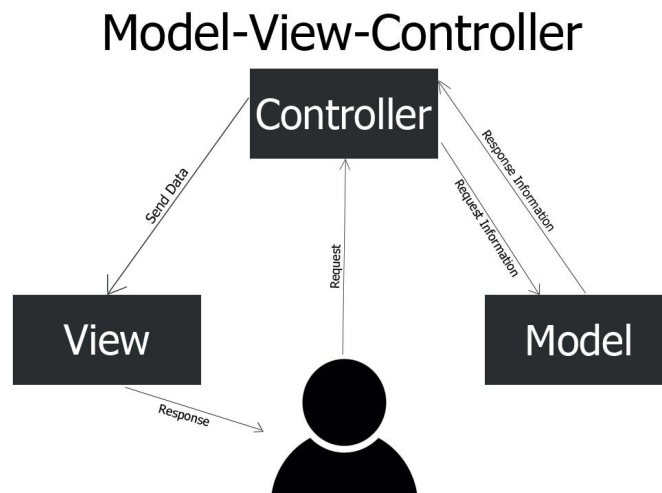
Questa organizzazione modulare permette una gestione chiara e scalabile del codice, semplificando l'aggiornamento e la manutenzione dell'applicazione.

2.4 Laravel

Laravel è un framework PHP open-source progettato per semplificare lo sviluppo di applicazioni web. Creato da Taylor Otwell nel 2011, Laravel ha guadagnato rapidamente popolarità grazie alla sua sintassi elegante, alla ricca documentazione e alla robusta architettura. È stato pensato per facilitare la gestione di operazioni comuni come la gestione delle rotte, l'autenticazione degli utenti, il trattamento delle richieste HTTP e la gestione dei database, riducendo la complessità del codice.

Una delle caratteristiche distintive di Laravel è il suo *Eloquent ORM* (Object-Relational Mapping), che permette una gestione semplice e intuitiva dei dati in un database relazionale. Inoltre, Laravel include funzionalità avanzate come la gestione delle code di lavoro, il supporto per le migrazioni di database e il sistema di notifiche.

Laravel è strutturato per essere altamente modulare, consentendo agli sviluppatori di costruire applicazioni scalabili e mantenibili con facilità. La sua architettura basata su MVC (Model-View-Controller) aiuta a separare la logica di presentazione dalla logica applicativa, migliorando la leggibilità e la manutenzione del codice.



In questa sezione, vedremo i principali vantaggi e componenti che sono inclusi nell'utilizzo di Laravel.

2.4.1 Sail

Laravel Sail è un ambiente di sviluppo basato su Docker che semplifica l'installazione e la gestione di un'applicazione Laravel in un container isolato. Con Sail, non è necessario configurare manualmente un ambiente di sviluppo locale, poiché tutto il necessario è già preconfigurato. Questo approccio riduce notevolmente la complessità dell'installazione di servizi come il database, Redis e la coda di lavoro, rendendo più semplice e veloce l'avvio di un nuovo progetto.

Nel contesto del rifacimento del gestionale interno, Sail si è rivelato uno strumento prezioso. Grazie alla sua integrazione con Docker, è stato possibile creare un ambiente di sviluppo replicabile e facilmente configurabile per tutti i membri del team. Ogni sviluppatore, infatti, può eseguire l'applicazione Laravel con un semplice comando, garantendo che l'ambiente di sviluppo sia omogeneo, riducendo il rischio di errori derivanti da differenze nelle configurazioni locali.

Utilizzando Sail, è stato possibile gestire in modo semplice i vari servizi necessari al funzionamento del gestionale, come il database MySQL, Redis per la gestione delle sessioni e altri servizi cruciali per il sistema.

2.4.2 Sanctum

Laravel Sanctum è una libreria che fornisce un sistema di autenticazione semplice e sicuro per le applicazioni Laravel, pensato per gestire sia l'autenticazione di sessione che l'autenticazione tramite API. Una delle caratteristiche principali di Sanctum è la sua capacità di generare e gestire token di autenticazione per gli utenti che desiderano accedere a determinate risorse protette all'interno dell'applicazione. A differenza di altre soluzioni, Sanctum sfrutta un sistema di token basati su CSRF (Cross-Site Request Forgery), che aiuta a proteggere le applicazioni da attacchi di tipo CSRF, garantendo che le richieste provenienti da client non autorizzati vengano bloccate.

Nel caso del gestionale interno di Dieffetech, Sanctum è stato utilizzato per implementare un sistema di login sicuro. Gli utenti, una volta autenticati tramite la piattaforma, ricevono un token che permette di accedere a specifiche risorse senza dover reinserire le credenziali. Questo sistema di token CSRF protegge ogni richiesta, assicurando che solo gli utenti con un token valido possano interagire con le API e accedere a risorse sensibili.

L'utilizzo di token CSRF è stato fondamentale per garantire una gestione sicura degli accessi, permettendo a ciascun dipendente di operare in modo sicuro e isolato all'interno dell'applicazione. Inoltre, Sanctum si integra facilmente con altre funzionalità di Laravel, come il middleware, per garantire che solo gli utenti autenticati possano accedere a determinate aree del gestionale, senza rischio di esposizione a vulnerabilità CSRF.

2.4.3 Telescope

Laravel Telescope è uno strumento di debugging e monitoraggio che consente di tracciare in tempo reale tutte le richieste HTTP, i log, gli errori, le eccezioni, i job della coda e altro ancora. È un'applicazione dedicata che offre un'interfaccia utente ricca di funzionalità per monitorare e diagnosticare le performance e il comportamento di un'applicazione Laravel durante il processo di sviluppo.

Nel contesto del rifacimento del gestionale, Telescope ha giocato un ruolo cruciale nel monitoraggio delle attività. La capacità di Telescope di tracciare ogni richiesta applicativa ha permesso di analizzare in tempo reale i dettagli operativi, individuando facilmente eventuali anomalie e ottimizzando il sistema. Telescope ha agevolato la risoluzione dei problemi di performance, consentendo una chiara visualizzazione delle query al database, incluse quelle duplicate, facilitando l'individuazione di aree ottimizzabili. La visibilità sulle risorse utilizzate e sulle lentezze nelle operazioni ha permesso di intervenire tempestivamente per ottimizzare il gestionale.

Grazie a questa potente funzionalità di monitoraggio, è stato possibile migliorare significativamente le performance del gestionale, garantendo un'esperienza utente più fluida e reattiva, e ottimizzando il sistema nel suo complesso.

2.5 Generazione del CRUD

Un CRUD è un acronimo che sta per Create, Read, Update, Delete, che rappresenta le quattro operazioni fondamentali per la gestione dei dati in un'applicazione. Queste operazioni consentono la gestione completa dei dati all'interno di un'applicazione:

- **Create:** Creazione di un nuovo record in un database.
- **Read:** Lettura dei dati da un database.
- **Update:** Modifica o aggiornamento di un record nel database.
- **Delete:** Eliminazione di un record da un database.

Prima di iniziare la creazione del CRUD, è essenziale conoscere le convenzioni di nomenclatura (come camelCase o snake_case, singolare o plurale) poiché i comandi e le funzioni standard di Laravel funzionano correttamente solo se si rispettano tali convenzioni.

Nel contesto del gestionale interno, i CRUD possono essere distinti in due livelli:

- **Primo livello:** Si riferisce ai modelli che vivono di vita propria, come un *progetto*. Questi modelli sono indipendenti e possono esistere senza dipendere da altri modelli.
- **Secondo livello:** Si riferisce ai modelli che esistono solo come figli di altri modelli, come le *task* di un progetto. Le task, infatti, esistono solo se esiste un progetto a cui sono associate.

Laravel offre una serie di comandi Artisan che permettono di generare automaticamente i file necessari per la creazione di un CRUD. Questi comandi generano i seguenti file:

- **Model:** Il modello rappresenta la struttura dei dati e va modificato aggiungendo i campi `fillable`, `sortable`, `filterable`, `searchable`, le relazioni e, se necessario, `fillableMedia`.
- **Migration:** La migration serve per definire la struttura della tabella del database e aggiungere i campi necessari.
- **Controller:** Il controller implementa le logiche per la creazione, modifica, lettura ed eliminazione dei dati.
- **Policy:** La policy di default viene generata con un `return true` per ogni azione standard, ma andrà modificata per definire i relativi permessi.
- **Resource:** Un file necessario per far funzionare Laravel correttamente, che non deve essere modificato.
- **Form Request:** Un file vuoto che va completato con tutti i campi che richiedono validazione.

Generalmente, in Laravel, esistono comandi specifici come `php artisan make:controller NomeController`, `php artisan make:model NomeModello`, ecc., che vengono utilizzati per creare i vari componenti di un'applicazione. Tuttavia, grazie alla creazione di comandi personalizzati, è possibile automatizzare l'intero processo di generazione del CRUD, velocizzando significativamente lo sviluppo del progetto.

Di seguito sono elencati i comandi di generazione che abbiamo creato, utilizzati per creare in modo rapido i file necessari.

Per il backend:

- **Primo livello:**

- `sail artisan app:crud {name}`
- Esempio: `sail artisan app:crud Project`

- **Secondo livello:**

- `sail artisan app:crud {name} --parent={name}`
- Esempio: `sail artisan app:crud Task --parent=Project`

Per il frontend:

- **Primo livello:**

- `sail artisan app:crud-fe {name}`
- Questo comando genera tre file principali:
 - * **Pagina:** Una pagina che include il componente della tabella.
 - * **Tabella:** Un componente `CrudDataTable` già completo delle props `title`, `form` e `url`. A questo componente vanno aggiunte le colonne necessarie.
 - * **Form:** Un file che conterrà solo il form, con i vari campi da aggiungere.
- Per rendere la nuova sezione raggiungibile, è necessario aggiungere l'apposita voce nel menu di `apps/react/src/menu/baseMenu.tsx`.

- **Secondo livello:**

- I CRUD nested sono utili quando si lavora con entità annidate. Per esempio, quando all'interno di un progetto esiste una tab `task` che contiene tutte le task associate al progetto.
- Comando: `sail artisan app:crud-fe {name} --parent={parent}`
- Esempio: `sail artisan app:crud-fe Task --parent=Project`

Questi comandi generano i componenti React per lavorare con le route annidate di Laravel, come nel caso in cui una task dipenda da un progetto. Questo comando predispone già il form e la `CrudDataTable` per lavorare con la struttura gerarchica delle route.

È fondamentale seguire la convenzione delle route e dei nomi delle risorse durante la creazione dei componenti, affinché la comunicazione tra frontend e backend funzioni correttamente.

2.6 API

Le API (Application Programming Interfaces) sono un insieme di definizioni e protocolli che permettono a diverse applicazioni software di interagire tra loro. Le API definiscono le modalità attraverso cui le informazioni vengono inviate e ricevute tra un client (ad esempio, un'applicazione front-end) e un server (che ospita la logica applicativa e i dati). Le API possono essere utilizzate per esporre funzionalità di un'applicazione in modo che possano essere utilizzate da altre applicazioni.

Nel contesto delle applicazioni web moderne, le API REST (Representational State Transfer) sono tra le più comuni. Le API REST operano attraverso le rotte HTTP standard (GET, POST, PUT, DELETE) per eseguire operazioni CRUD (Create, Read, Update, Delete) sui dati. Ogni rotta API è associata a un'azione, come l'aggiunta di un nuovo record o la lettura di uno esistente.

2.6.1 Le rotte API nel progetto

Le rotte API sono definite nel file di routing del progetto `routes/api.php`. Ogni rotta è associata a un controller specifico, incaricato della gestione della logica di business per la risorsa corrispondente. All'interno del progetto, sono presenti numerose rotte, ciascuna dedicata a uno o più modelli. Il codice 2.1 mostra le principali rotte API implementate nel corso dello sviluppo.

```
// Referenti
Route::api('referents', ReferentController::class);
// Appuntamenti
Route::api('appointments', AppointmentController::class);
// Offerte
Route::api('offers', OfferController::class);
// Task
Route::api('tasks', TaskController::class);
// Clienti
Route::api('customers', CustomerController::class);
Route::api('customers.referents', ReferentController::
    class);
Route::api('customers.appointments',
    AppointmentController::class);
Route::api('customers.offers', OfferController::class);
Route::api('customers.tasks', TaskController::class);
```

Codice 2.1: API

Solitamente, le rotte in Laravel vengono dichiarate con metodi come `Route::get`, `Route::post`, e così via. Tuttavia, nel nostro progetto abbiamo creato una rotta personalizzata `Route::api`, che consente di generare automaticamente tutte le rotte necessarie (come `index`, `store`, `show`, `update`, `destroy`) per ogni risorsa con una singola dichiarazione.

2.6.2 Funzionamento delle rotte

Le rotte dichiarate nel file `routes/api.php` definiscono i punti di accesso per il client per interagire con diverse risorse del sistema:

- `Route::api('referents', ReferentController::class);` definisce una rotta per gestire le operazioni sui referenti. La rotta sarà gestita dal `ReferentController`.
- `Route::api('appointments', AppointmentController::class);` definisce una rotta che consente di interagire con gli appuntamenti attraverso il `AppointmentController`.
- `Route::api('offers', OfferController::class);` permette la gestione delle offerte tramite il `OfferController`.
- `Route::api('tasks', TaskController::class);` definisce una rotta per la gestione dei task, utilizzando il `TaskController`.
- `Route::api('customers', CustomerController::class);` è la rotta principale per interagire con i clienti, gestita dal `CustomerController`.
- Le rotte con il prefisso `customers` (come `customers.referents`, `customers.appointments`, `customers.offers`, `customers.tasks`) definiscono le relazioni tra i clienti e le rispettive risorse. Ad esempio, `customers.referents` permette di ottenere o gestire i referenti associati a un cliente, `customers.appointments` gestisce gli appuntamenti di un cliente, e così via.

Queste rotte sono associate ai rispettivi controller e sono utilizzate per eseguire operazioni come il recupero, la creazione, l'aggiornamento o la cancellazione dei dati tramite richieste HTTP. Ogni controller gestisce una specifica risorsa e si occupa di applicare la logica di business necessaria per rispondere alle richieste del client.

Inoltre, tutte le rotte API sono avvolte dal middleware `throttle:api`, che limita il numero di richieste al minuto per evitare attacchi di brute forcing, proteggendo il sistema da un sovraccarico di richieste non autorizzate.

Insieme al `throttle:api`, viene utilizzato anche il middleware `auth:sanctum`, di cui è stata precedentemente trattata la funzione, per garantire che solo utenti autenticati possano interagire con le API, assicurando così che ogni richiesta provenga da un utente validamente autenticato.

```
Route::middleware('throttle:api', 'auth:sanctum')->
    group(function() {
        Route::api('route', RouteController::class)
        ...
        ...
    })
```


Capitolo 3

Contributo Personale

La ristrutturazione del gestionale ha visto un intervento che si è concentrato prevalentemente sull'area relativa alla gestione dei clienti. In particolare, sono state apportate diverse modifiche e miglioramenti per ottimizzare il flusso di lavoro e la gestione delle informazioni.

Un aspetto chiave delle modifiche riguarda l'espansione delle funzionalità esistenti per consentire una gestione più dettagliata e completa dei dati relativi ai clienti. Tra le principali novità, sono state introdotte nuove sezioni che permettono di monitorare e gestire con maggiore precisione:

- **Leads:** per tracciare le opportunità di vendita e monitorare l'avanzamento di ogni potenziale cliente.
- **Appuntamenti:** per organizzare e tenere traccia degli incontri e delle interazioni con i clienti.
- **Offerte:** per gestire le proposte commerciali e monitorarne lo stato di avanzamento.
- **Task:** per la gestione delle attività associate a ciascun cliente, migliorando l'assegnazione e il monitoraggio delle azioni da intraprendere.

Questi miglioramenti hanno reso il gestionale più versatile, consentendo agli utenti di avere una visione più completa e aggiornata delle attività legate ai propri clienti. L'introduzione di queste sezioni ha contribuito a migliorare l'efficienza operativa, facilitando la pianificazione e l'esecuzione delle attività quotidiane.

3.1 Prima e ora

Prima delle modifiche introdotte, la gestione dei clienti nel sistema risultava piuttosto semplice e centralizzata. Ogni cliente veniva trattato come un'entità unica, senza una distinzione tra i diversi stadi del processo di acquisizione o gestione. In pratica, i dati relativi ai clienti erano registrati e archiviati senza un livello di dettaglio che permettesse di tracciare l'evoluzione di un cliente dalla fase iniziale di interesse fino alla finalizzazione di un contratto o di un progetto.

Il sistema era strutturato per gestire unicamente le informazioni base dei clienti, come nome, contatti e dettagli generali. Non c'era una chiara separazione tra i clienti già acquisiti e i potenziali clienti, i cosiddetti *Leads*, né una gestione dei task o delle offerte. Questo rendeva difficile monitorare lo stato di avanzamento delle trattative, con una gestione che si limitava a un archivio statico delle informazioni.

Le modifiche apportate hanno trasformato significativamente il modo in cui il sistema gestisce i clienti, rendendo il flusso di lavoro più dinamico e interattivo. In particolare, è stata introdotta una separazione tra i **Leads** e i clienti acquisiti. Ora, ogni potenziale cliente (Lead) può essere tracciato separatamente, permettendo una gestione più precisa e una visibilità migliore sullo stato delle trattative.

Le nuove funzionalità permettono di:

- Monitorare i **Leads**, che rappresentano i potenziali clienti ancora nella fase di acquisizione.
- Gestire gli **Appuntamenti**, per programmare e tenere traccia delle interazioni con i clienti/leads.
- Creare e gestire **Offerte**, per monitorare le proposte fatte ai clienti/leads e il loro stato.
- Gestire i **Task** associati a ciascun cliente/lead, facilitando l'assegnazione e la pianificazione delle attività.

Queste modifiche hanno reso il sistema più fluido, con la possibilità di avere una visione completa delle opportunità di vendita e delle attività in corso. La divisione tra Leads e clienti acquisiti ha consentito una gestione più efficiente delle priorità e ha migliorato la pianificazione delle risorse. Di seguito la tabella visualizzata:

3.2 Tabella dei Clienti e Leads

Per la gestione unificata di Clienti e Leads, è stata utilizzata una singola tabella nel database, distinguendo le due tipologie attraverso la colonna `is_customer`. Questa colonna assume un valore booleano, dove `true` indica un Cliente e `false` identifica un Lead.

Ragione sociale	Partita IVA	Settore	Sito Web	Stato	Data creazione
Lead0	00	Videoludico	0.com	Non interessato	24/01/2025
Lead1	01	Industriale	1.com	Contattato	24/01/2025
Lead2	02	Informatico	2.com	Da contattare	24/01/2025
Lead3	03	Biologico	3.com	Non interessato	24/01/2025
Lead4	04	Stradale	4.com	Scartato	24/01/2025
Lead5	05	Scolastico	5.com	Da contattare	24/01/2025
Lead6	06	Statale	6.com	Da contattare	24/01/2025
Lead7	07	Farmaceutico	7.com	Non interessato	24/01/2025
Lead8	08	Sportivo	8.com	Contattato	24/01/2025
Lead9	09	Aereonautica	9.com	Contattato	24/01/2025

Figura 3.1: Tabella dei Leads.

Nella figura 3.1 è possibile osservare una visualizzazione che mostra esclusivamente i Leads. Ciò avviene grazie al valore della colonna `is_customer` passato come parametro alle props di `CrudDataTable`. In particolare, questo parametro viene gestito all'interno della proprietà `params`. Quando si visualizza la tabella dei Clienti, `is_customer` è impostato a `true`, mentre nella tabella dei Leads è impostato a `false`.

3.2.1 Configurazione della `CrudDataTable`

Il codice relativo alla `CrudDataTable`, presenta numerose proprietà che regolano il comportamento e la struttura della tabella:

- **url**: esegue la chiamata fetch al database per ottenere i dati da visualizzare.

- **form**: definisce il form di creazione, che viene visualizzato al click sul pulsante **+Aggiungi**.
- **params**: Sono i parametri che vengono passati in backend, aggiunti come filtri di ricerca.
- **columns**: specifica l'elenco delle colonne da visualizzare nella tabella, con:
 - **title**: il nome della colonna.
 - **dataIndex**: il campo della tabella su cui si basa la colonna.
 - **detail/sorter/search**: attributi booleani che definiscono se la colonna può essere filtrata, ordinata o ricercata.
- **tabs**: Gestisce le sezioni di dettaglio visualizzabili al click su un elemento della tabella. Le tabs contengono **items**, ciascuno con:
 - **label**: il nome della sezione.
 - **key**: l'identificativo univoco della tab.
 - **children**: il contenuto visualizzato all'interno della tab (a sua volta una `CrudDataTable`).

```
export const CustomersTable = () => {
  const { t } = useTranslation();

  return (
    <CrudDataTable
      url={"/customers"}
      form={CustomerForm}
      params={{
        is_customer: true,
      }}
      tabs={(item) => {
        return [
          {
            label: t("Referenti"),
            key: "referents",
            children: (
              <ReferentTable url={`'/customers/${item
?.id}/referents`} />
            ),
          },
          {
```

```

        label: t("Appuntamenti"),
        key: "appointments",
        children: (
            <AppointmentsTable url={`/${customers/${
item?.id}/appointments`} />
        ),
    },
    {
        label: t("Task"),
        key: "customer-tasks",
        children: (
            <CustomerTasksTable url={`/${customers/${
item?.id}/customer-tasks`} />
        ),
    },
    {
        label: t("Offerte"),
        key: "offers",
        children: (
            <CustomerOffersTable url={`/${customers/${
{item?.id}/offers`} />
        ),
    },
    ];
}}
columns=[
    {
        title: t("Ragione sociale"),
        dataIndex: "business_name",
        detail: true,
        sorter: true,
        search: true,
    },
    ...
]
/>
);
};

```

Codice 3.1: Customers Table Code

Una visione più dettagliata del comportamento delle tabs può essere osservata nel codice riportato in 3.1. Quando si clicca sul dettaglio di un Cliente o Lead, vengono visualizzate diverse sezioni: *Referenti*, *Appuntamenti*, *Task* e *Offerte*.

Ognuna di queste sezioni è rappresentata da una tabella dedicata, che viene popolata tramite una chiamata API, la quale restituisce i dati specifici del Cliente o Lead selezionato.

La chiamata API utilizza il seguente formato per i parametri: `/customers/item?.id/key`, dove `item?.id` rappresenta l'ID univoco del Cliente o Lead attualmente selezionato, e `key` identifica la sezione richiesta (ad esempio, *referents*, *appointments*, *tasks* o *offers*).

Nel caso specifico della colonna "Ragione Sociale", notiamo che il parametro `detail` è impostato su `true`. Questo significa che il nome visualizzato nella colonna viene reso interattivo (come si può osservare nella tabella in Figura 3.1, dove i nomi sono evidenziati in azzurro). Cliccando su uno di questi nomi, si apre la schermata dei dettagli del Cliente o Lead (3.2), in cui è possibile accedere e navigare tra le varie sezioni di dettaglio.

Home / Leads

Dettaglio Referenti Appuntamenti Task Offerte

Anagrafica Lead

* Ragione sociale: Lead0

Partita IVA: 00

Codice fiscale: 222222222222222222

Settore: Videoludico

Codice ATECO: 11.11.11

Fatturato: 5000000

IVA: 0 %

Sito web: 0.com

* Generato da: Administrator Dieffetech

* Origine: origine

* Stato: Non interessato

Qualità lead: 5

Note: Note per il lead

Indirizzo

* Indirizzo: Via Prova 1

* Città: Città

* Provincia: CT

* CAP: 00000

* Nazione: Mondo

Indietro Salva

Figura 3.2: Dettagli di un Lead.

3.3 Appuntamenti

In questa sezione vengono gestiti gli appuntamenti, visualizzati e organizzati all'interno di una tabella specifica, che può essere vista nella Figura 3.3. La tabella è parte della componente `CrudDataTable`, la quale consente la gestione

dinamica dei dati. Ogni colonna della tabella è definita da proprietà come `title`, `dataIndex`, e altre opzioni che permettono di abilitare il filtraggio o l'ordinamento.

La tabella degli appuntamenti ha collegamenti con altre entità del sistema, tra cui la tabella dei referenti (che permette di selezionare un referente specifico per ogni appuntamento, associato al cliente selezionato) e la tabella degli utenti (che permette di associare un utente registrato nel sistema, all'appuntamento). Questi collegamenti vengono gestiti attraverso le relazioni **Eloquent** definite nel Model di **Appointments**. Queste relazioni permettono di navigare facilmente tra i dati correlati e semplificano la gestione delle associazioni all'interno del database.

Titolo	Data e ora	Referente	Tipologia	Associato a	Descrizione	Stato	Feedback
Appuntamento1	01/01/2025	Referente 1	Call	Administrator Dieffetech	Descrizione Appuntamento1	Programmato	Feedback Appuntamento1
Appuntamento2	02/01/2025	Referente 2	Call	Administrator Dieffetech	Descrizione Appuntamento2	Annullato	Feedback Appuntamento2

Figura 3.3: Tabella degli appuntamenti.

Oltre alla tabella, la gestione degli appuntamenti include anche la possibilità di creare nuovi appuntamenti attraverso un form dedicato, visualizzato nella Figura 3.4. Questo form consente di compilare i campi necessari per l'inserimento di un nuovo appuntamento, tra cui il nome del referente e l'utente associato. Ogni campo del form è definito all'interno del componente **AppointmentsForm**, che si occupa di validare i dati inseriti dall'utente e inviarli al backend per il salvataggio nel database. Anche qui, i campi collegati

CAPITOLO 3. CONTRIBUTO PERSONALE

con altre entità, come i referenti e gli utenti, vengono popolati dinamicamente grazie alle relazioni **Eloquent**.

The screenshot shows a web application interface for managing appointments. A modal form titled 'Appuntamenti' is open, allowing users to create or edit an appointment. The form includes several input fields and dropdown menus: 'Titolo' (Text), 'Data e Ora' (Date/Time), 'Referente' (Dropdown), 'Tipologia' (Dropdown), 'Associato a' (Dropdown), 'Stato' (Dropdown), 'Feedback' (Text), 'Descrizione' (Text), and 'Allegati' (File Upload). The 'Allegati' section has a dashed border and a file icon, with instructions: 'Clicca o trascina e rilascia qui un file. Formati accettati: .jpeg, .jpg, .png, .pdf'. At the bottom of the modal are 'Indietro' and 'Salva' buttons. The background shows a sidebar with 'Leads' and a table of appointments with columns for 'Feedback', 'Appuntamento1', and 'Appuntamento2'. The table has two rows of data and a pagination bar at the bottom indicating '2 di 2 elementi' and '20 / pagina'.

Figura 3.4: Form degli appuntamenti.

Entrando nel Model degli Appuntamenti, infatti, possiamo osservare che nella sezione dedicata alle funzioni di relazione troviamo il seguente codice:

```
public function customer(): BelongsTo
{
    return $this->belongsTo(related: Customer::class);
}
public function referent(): BelongsTo
{
    return $this->belongsTo(related: Referent::class);
}
public function userAdmin(): BelongsTo
{
    return $this->belongsTo(related: UserAdmin::class);
}
```


3.4 Task

In questa sezione vengono gestiti i task, che sono simili agli appuntamenti nella loro organizzazione, ma presentano una differenza importante: ogni task è associato a un appuntamento specifico e ad un utente, mentre gli appuntamenti erano legati a un referente e a un utente.

I task sono visualizzati e organizzati all'interno di una tabella, come mostrato nella Figura 3.5, utilizzando le stesse modalità degli appuntamenti, con la componente `CrudDataTable` e le relative proprietà per la gestione dinamica dei dati.

Un aspetto distintivo dei task è la relazione con la tabella degli appuntamenti, che consente di associare ogni task a un appuntamento specifico. Inoltre, come per gli appuntamenti, i task sono legati agli utenti, permettendo di assegnare un determinato compito a una persona all'interno dell'azienda. Queste relazioni vengono gestite tramite le definizioni `Eloquent` nel `Model Task`.

Titolo	Descrizione	Assegnato a	Data	Note	Appuntamento collegato
Task1	Descrizione Task1	Administrator Dieffetech	04/01/2025	Note Task1	Appuntamento1
Task2	Descrizione Task2	Administrator Dieffetech	07/01/2025	Note Task2	Appuntamento2

Figura 3.5: Tabella dei task.

Oltre alla tabella, la gestione dei task include la possibilità di creare nuovi task tramite un form dedicato, mostrato nella Figura 3.6. Il form consente di selezionare l'appuntamento associato e gli utenti coinvolti nel task. Ogni campo è definito nel componente `TaskForm`, che gestisce la validazione dei dati

e l'invio al backend per il salvataggio nel database. Le relazioni tra i task, gli appuntamenti e gli utenti sono gestite dinamicamente grazie a **Eloquent**.

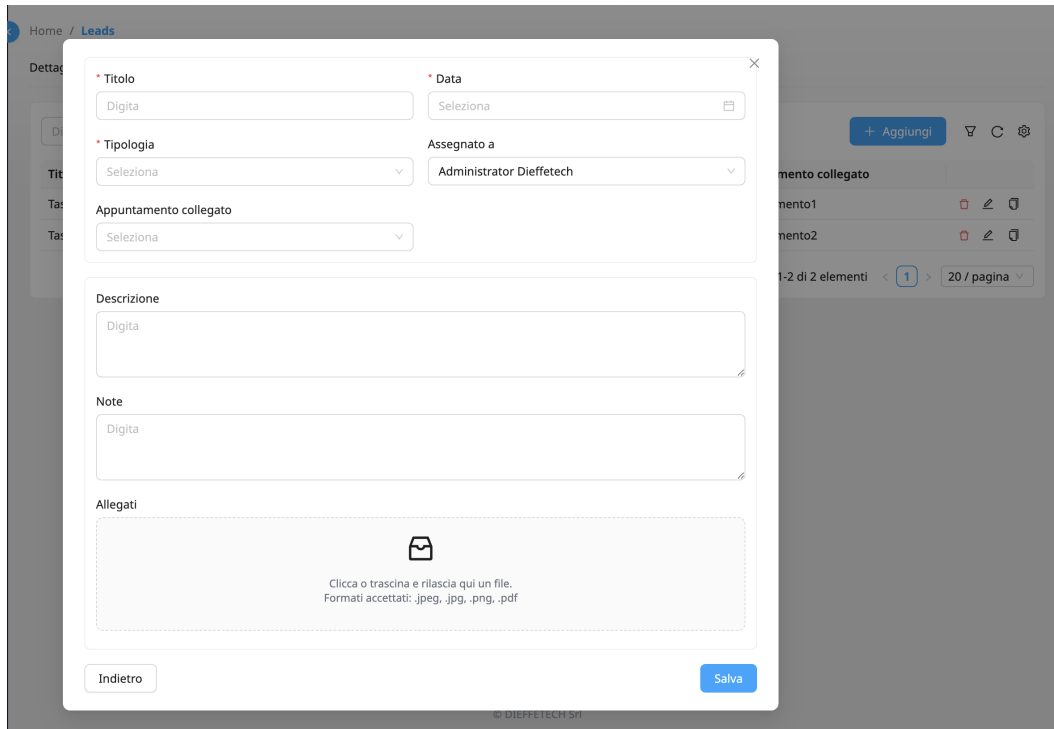


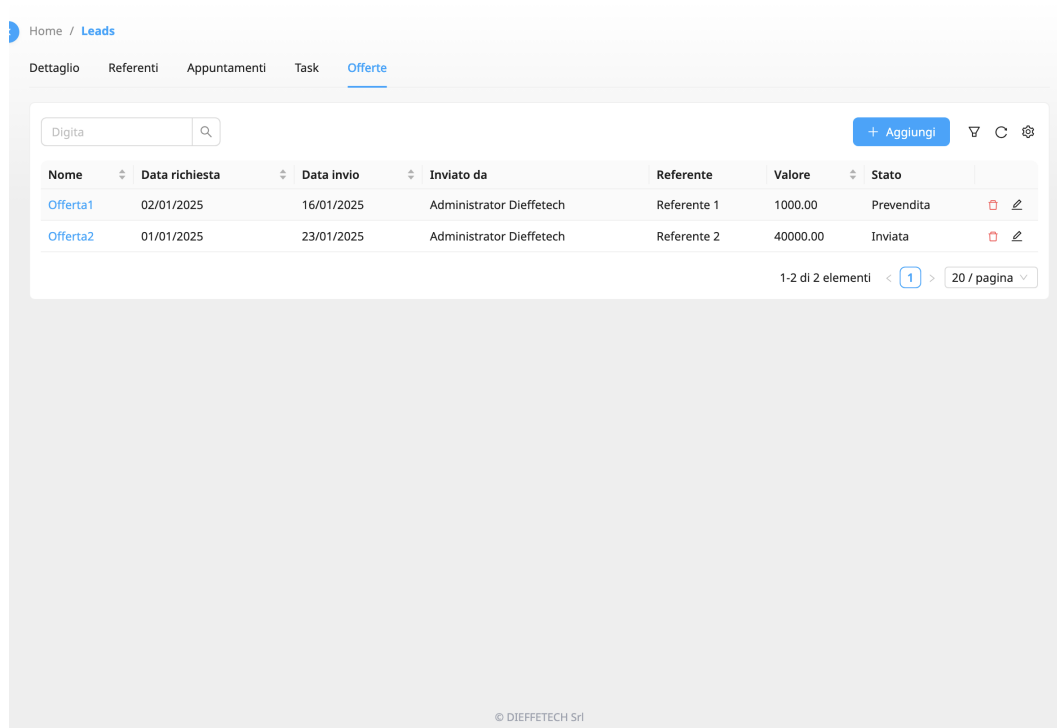
Figura 3.6: Form dei task.

Nel Model dei Task, possiamo osservare che nella sezione dedicata alle funzioni di relazione troviamo il seguente codice:

```
public function customer(): BelongsTo
{
    return $this->belongsTo(related: Customer::class);
}
public function appointment(): BelongsTo
{
    return $this->belongsTo(related: Appointment::class);
}
public function userAdmin(): BelongsTo
{
    return $this->belongsTo(related: UserAdmin::class);
}
```

3.5 Offerte

La sezione dedicata alle **offerte** merita un'attenzione particolare, in quanto presenta una logica più complessa rispetto alle altre sezioni. Inoltre, è l'unica ad avere una sezione dedicata nel baseMenu.



Nome	Data richiesta	Data invio	Inviato da	Referente	Valore	Stato
Offerta1	02/01/2025	16/01/2025	Administrator Dieffetech	Referente 1	1000.00	Prevendita
Offerta2	01/01/2025	23/01/2025	Administrator Dieffetech	Referente 2	40000.00	Inviata

Figura 3.7: Tabella delle offerte.

A prima vista, la tabella potrebbe sembrare simile alle altre, e in effetti, in termini di componenti, lo è. Si tratta infatti di una **CrudDataTable**, con il suo pulsante **+Aggiungi** che, al click, apre il form di inserimento.

CAPITOLO 3. CONTRIBUTO PERSONALE

Quando si accede ai dettagli di un'offerta (cliccando sull'icona della matita), si apre un form di modifica, visualizzato come un "drawer" (poiché viene aperto lateralmente), che consente di modificare i dettagli dell'offerta. Una funzionalità aggiuntiva in questa sezione è il pulsante **"Aggiungi Revisione"**.

The image shows a mobile application interface. On the left, a dark grey 'drawer' is open, displaying a table with offer details. The table has columns for 'Nome', 'Data richiesta', and 'Data invio'. It lists 'Offerta1' and 'Offerta2'. To the right of the drawer, a white form is visible. At the top right of the form is a blue button labeled 'Aggiungi revisione'. The form contains several input fields: 'Nome' (Offerta1), 'Data richiesta' (02/01/2025), 'Stato' (Prevendita), 'Referente' (Referente 1), 'Data di invio' (16/01/2025), and 'Inviato da' (Administrator Dieffetech). Below these is a 'Valore offerta' field with '1.000' and a Euro symbol. At the bottom of the form is an 'Allegato' section with a file upload icon and text: 'Clicca o trascina e rilascia qui un file. Formati accettati: jpeg, jpg, png, pdf'. At the bottom of the drawer, there are 'Indietro' and 'Salva' buttons.

Figura 3.8: Dettaglio di un'offerta.

Al clic, viene visualizzato un form aggiuntivo per la creazione di una revisione (di tipo "modal", in quanto ha il suo riquadro di compilazione in posizione centrale dello schermo). Una volta registrata, la revisione diventa un "children" dell'offerta padre e viene visualizzata in tabella al posto dell'offerta originaria.

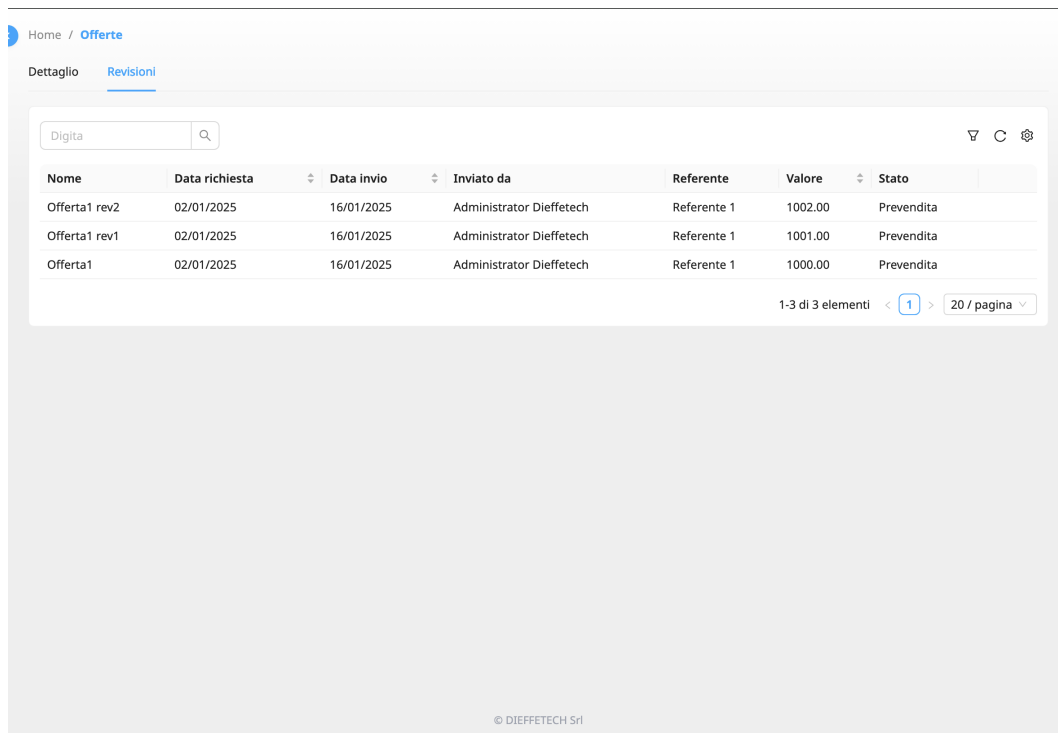
The image shows a modal window titled 'Modale di revisione' overlaid on the previous screen. The modal contains a form for creating a revision. It includes fields for 'Nome' (Offerta1 rev3), 'Data richiesta' (02/01/2025), 'Stato' (Prevendita), 'Referente' (Referente 1), 'Data di invio' (16/01/2025), and 'Inviato da' (Administrator Dieffetech). There is also a 'Valore offerta' field with '1.002' and a Euro symbol, and an 'Offerta Padre' field with '18'. At the bottom of the modal is an 'Allegato' section with a file upload icon and text: 'Clicca o trascina e rilascia qui un file. Formati accettati: jpeg, jpg, png, pdf'. A blue 'Salva' button is at the bottom right of the modal. The background shows the same drawer and form as in Figure 3.8, but they are dimmed.

Figura 3.9: Modale di revisione.

Cliccando sul nome dell'offerta, si accede alla pagina delle offerte, visualizzando i dettagli di quella selezionata. In questa vista, si possono notare due sezioni:

- **Dettaglio:** il dettaglio dell'offerta selezionata.
- **Revisioni:** l'elenco di tutti gli stati precedenti dell'offerta, a partire dall'offerta iniziale fino a tutte le altre revisioni precedenti.

Nella sezione delle revisioni, avremo un elenco di tutti gli stati precedenti dell'offerta selezionata. Ponendo il caso di un'offerta con 3 revisioni, avremo l'offerta originale, la prima revisione e la seconda revisione.



Home / Offerte

Dettaglio Revisioni

Digita

Nome	Data richiesta	Data invio	Inviato da	Referente	Valore	Stato
Offerta1 rev2	02/01/2025	16/01/2025	Administrator Dieffetech	Referente 1	1002.00	Prevendita
Offerta1 rev1	02/01/2025	16/01/2025	Administrator Dieffetech	Referente 1	1001.00	Prevendita
Offerta1	02/01/2025	16/01/2025	Administrator Dieffetech	Referente 1	1000.00	Prevendita

1-3 di 3 elementi < 1 > 20 / pagina

© DIEFFETECH Srl

Figura 3.10: Revisioni di un'offerta.

3.5.1 Offer Model

Per comprendere appieno i meccanismi di visualizzazione delle offerte e delle revisioni, è necessario analizzare il Model delle Offerte.

```
class Offer extends BaseModel
{
  use Log;

  protected $fillable = [
    'code',
    'name',
    'offer_value',
    ...
  ];

  protected $fillableMedia = [
    'attachment',
  ];

  protected $defaultSort = 'name';

  public function sortable(): array
  {
    return [
      'name',
      'offer_value',
      ...
    ];
  }

  public function filterable(): array
  {
    return [
      'name',
      'offer_value',
      AllowedFilter::callback('
has_no_children', function ($query
) {
    $query->whereNotIn('id',
function ($subQuery) {
    $subQuery
    ->select('offer_parent_id'
)
    ->from('offers')
    ->whereNotNull('
offer_parent_id');
    });
    },
    ...
  ];
}

protected $searchable = [
  'name',
  ...
];

// Relazioni
public function customer():
  BelongsTo
  {
    return $this->belongsTo(Customer::
class)
  }

public function userAdmin():
  BelongsTo
  {
    return $this->belongsTo(UserAdmin
:: class)
  }

public function referent():
  BelongsTo
  {
    return $this->belongsTo(Referent::
class)
  }

public function parentOffer():
  BelongsTo
  {
    return $this->belongsTo(Offer::
class, 'offer_parent_id')->with('
userAdmin', 'referent', '
parentOffer');
  }

public function childrenOffer():
  HasMany
  {
    return $this->hasMany(Offer:: class
, 'offer_parent_id')->with('
userAdmin', 'referent', '
childrenOffer');
  }

public function scopeHasNoChildren(
  Builder $query)
  {
    return $query->whereNotIn('id',
function ($subQuery) {
    $subQuery
    ->select('offer_parent_id')
    ->from('offers')
    ->whereNotNull('
offer_parent_id');
    });
  }
}
```

Figura 3.11: Model delle offerte

Ogni Model ha una struttura generale con diverse sezioni per gestire le varie funzionalità dell'applicazione:

- **\$fillable**: Contiene tutti i campi del database che possono essere assegnati. Nel caso delle offerte, sono presenti tutti i campi necessari per rappresentare un'offerta nel database.
- **\$fillableMedia**: Gestisce gli allegati associati all'offerta, come file o media. Approfondiremo l'argomento nel capitolo 3.6.2.
- **\$defaultSort**: Definisce la colonna di ordinamento predefinito per la visualizzazione delle offerte. Questo determina quale campo verrà usato per ordinare le offerte nella vista tabellare.
- **sortable()**: Restituisce le colonne che possono essere utilizzate per il riordinamento delle offerte. Consente una gestione dinamica dell'ordinamento attraverso il frontend, migliorando la flessibilità dell'interfaccia utente.
- **filterable()**: Definisce le colonne che possono essere filtrate dal frontend. Questa funzione include anche filtri avanzati come `AllowedFilter::callback()`, che ad esempio può escludere le offerte che hanno delle revisioni (offerte figlie).
- **\$searchable**: Elenca tutte le colonne del database che possono essere ricercate dal frontend. Permette una funzionalità di ricerca rapida e dinamica per gli utenti.
- **Funzioni di Relazione**:
 - **customer()**: Definisce la relazione **BelongsTo** con il modello **Customer**. Rappresenta il legame tra l'offerta e il cliente a cui è associata, permettendo di accedere ai dettagli del cliente direttamente dal record dell'offerta.
 - **userAdmin()**: Definisce la relazione **BelongsTo** con il modello **User**. Questa relazione indica quale amministratore ha gestito o creato l'offerta, fornendo una connessione diretta ai dati dell'utente responsabile.
 - **referents()**: Definisce la relazione **BelongsTo** con il modello **Referent**. Indica quali referenti sono associati all'offerta, permettendo di accedere ai contatti all'interno dell'azienda cliente direttamente tramite l'offerta. Approfondiremo l'argomento nel capitolo 3.6.1.

- `parentOffer()`: Definisce la relazione `BelongsTo` con l’offerta padre. Questa relazione è utile per gestire le offerte che sono delle revisioni o delle varianti di un’offerta precedente.
- `childrenOffer()`: Definisce la relazione `HasMany` per gestire tutte le revisioni di un’offerta. Le offerte figlie sono visualizzate in tabella al posto dell’offerta padre una volta che vengono create.

- **Funzioni Scope:**

- `scopeHasNoChildren()`: Definisce uno scope per ottenere solo le offerte che non hanno revisioni associate. Viene utilizzato per filtrare e visualizzare nella tabella delle Offerte, solo le righe nel DB che non hanno nessun figlio associato, ovvero la revisione più recente.

- **Funzioni Personalizzate:**

- `generateCode()`: Una funzione personalizzata che genera un codice unico per ogni offerta, basato sull’ID del cliente e sull’ID dell’offerta padre. Questo codice serve a identificare in modo univoco ogni offerta e le sue eventuali revisioni.

3.5.2 Children e Parents

Le relazioni `Children` e `Parent` costituiscono relazioni ricorsive, in quanto entrambe sono definite all’interno del modello `Offer` e fanno riferimento alla medesima entità, ovvero l’offerta. Queste relazioni permettono di gestire la gerarchia tra le offerte e le loro revisioni.

- **Children:** Questa relazione tiene traccia delle offerte figlie di un’offerta esistente. Quando un’offerta viene creata, essa non ha né genitori né figli. Tuttavia, alla creazione di una revisione, l’offerta originale diventa il "genitore" e la revisione diventa il "figlio". In questo modo, si forma una catena di revisioni, che collega l’offerta originale (il genitore) a tutte le sue revisioni successive (i figli).

La catena si estende così: $Offerta \rightarrow Revisione1 \rightarrow \dots \rightarrow RevisioneX$, con ogni revisione che punta alla precedente e viceversa.

- **Parent:** Questa relazione permette di risalire dalla revisione corrente all’offerta originale (il genitore). Tramite questa struttura, è possibile mantenere un collegamento tra l’offerta iniziale e la sua ultima revisione, che sarà quella visibile nella tabella principale delle offerte.

Grazie a queste relazioni "a catena", è possibile ottenere sia l'offerta iniziale che la sua ultima revisione, garantendo una navigazione fluida tra le varie versioni di un'offerta. Questa struttura è anche essenziale per la sezione "Revisioni" presente nei dettagli dell'offerta, poiché permette di risalire lungo la catena delle revisioni, dall'ultima fino all'offerta da cui tutto è iniziato.

Un'altra caratteristica interessante è l'utilizzo della funzione `->with("attributi")`. Questa funzione assicura che, quando viene caricata un'offerta o una revisione (sia essa figlia o genitore), vengano caricati anche i relativi attributi e le rispettive relazioni. In questo modo, è possibile ottenere l'intera catena di relazioni *genitore-figlio*, dall'offerta originale fino all'ultima revisione, con un solo caricamento, evitando il cosiddetto "N + 1 query problem".

3.6 Morph Relations

Le **morph relations** sono un tipo particolare di relazioni in Laravel che permettono a un singolo modello di appartenere a più altri modelli su una singola associazione. A differenza delle relazioni tradizionali, in cui un modello appartiene sempre a un altro modello specifico (es. una relazione **hasOne** o **belongsTo**), le morph relations consentono una maggiore flessibilità e riutilizzabilità.

Queste relazioni utilizzano una coppia di campi nel database: uno per memorizzare l'ID dell'entità collegata, e l'altro per memorizzare il tipo del modello. Questo significa che un modello può essere associato a diversi modelli di tipo differente tramite la stessa relazione. Un esempio semplice, un modello **Comment** potrebbe appartenere a modelli **Post** o **Video**, senza bisogno di creare relazioni distinte.

Esistono due tipi principali di relazioni morph:

- **Relazione one-to-one o one-to-many morph:** permette a un modello di appartenere a uno o più modelli di tipo differente. Ad esempio, un commento può essere associato sia a un post che a un video.
- **Relazione many-to-many morph:** permette a un modello di appartenere a più modelli di tipo differente in una relazione many-to-many. Ad esempio, un tag può essere associato sia a post che a video.

Le morph relations sono estremamente utili per ridurre la duplicazione di codice e mantenere il database flessibile, specialmente quando si lavora con sistemi che richiedono l'associazione dinamica tra modelli di diversa natura.

3.6.1 Referenti

Ogni **Cliente** o **Lead** può avere più referenti, a ciascuno dei quali possono essere attribuiti più appuntamenti e offerte. Nella sezione *Referenti*, verrà visualizzato un elenco dei referenti esistenti per il cliente selezionato, con la possibilità di aggiungerne di nuovi tramite il form accessibile cliccando sul bottone **+Aggiungi**.

Nome	Cognome	Email	Ruolo
Referente	1	email@referente1.com	Referente
Referente	2	email@referente2.com	Referente

Figura 3.12: Tabella dei referenti.

Ogni cliente ha i propri referenti, e infatti, durante la compilazione di un appuntamento, nell'elenco dei referenti disponibili ci saranno solo quelli associati al cliente o lead selezionato.

The screenshot displays a web interface for managing 'Referenti' (Referents). A modal form is open for adding a new entry. The form fields are:

- * Nome (Name): Digita
- * Cognome (Surname): Digita
- E-mail: Digita
- Telefono (Telephone): Digita
- Ruolo (Role): Digita
- Tipologia contatto (Contact type): Digita

Buttons at the bottom of the form are 'Indietro' (Back) and 'Salva' (Save). The background shows a table with columns 'Ruolo', 'Referente', and 'Referente'. The table has 20 rows and is currently on page 1 of 20.

Figura 3.13: Form di inserimento per i referenti.

L'aggiunta di un nuovo referente avviene tramite il relativo form, che permette di inserire i dati del referente. Il "collegamento" del referente al cliente avviene tramite la relazione morph.

Una volta creato un nuovo referente, il database viene aggiornato con l'aggiunta di un record contenente i dati inseriti nel form. In particolare, le colonne `model_type` e `model_id` vengono automaticamente valorizzate in base alla posizione corrente. Nel caso specifico, dopo aver creato due referenti per il cliente con ID 26, sono stati inseriti due record con i dati del form, e automaticamente la colonna `model_type` è stata valorizzata con `App/Models/Customer` e `model_id` con il valore 26, che rappresenta l'ID del cliente.

123 id ▼	A-Z model_type ▼	123 model_id ▼
1	App\Models\Customer	1
2	App\Models\Customer	26
3	App\Models\Customer	26

Figura 3.14: Referente nel database.

3.6.2 Media

Nei form precedentemente discussi è presente una sezione denominata "allegati", che consente di caricare file immagine relativi all'appuntamento, all'offerta o al task selezionato.

La gestione dei file multimediali è affidata al componente `FilesUploader`.

```
<FilesUploader
  storage="private"
  label={t("Allegati")}
  name={["attachment"]}
  uploadProps={{ listType: "picture-card"}}
  multiple
  draggable
/>
```

Codice 3.2: Files Uploader

Come la `CrudDataTable`, il `FilesUploader` dispone di alcune proprietà, tra cui:

- `uploadProps`: Consente di configurare il tipo di anteprima per l'immagine caricata, con opzioni come `text`, `picture`, `picture-card` e `picture-circle`.
- `multiple`: Abilita o disabilita la possibilità di caricare più immagini contemporaneamente.
- `draggable`: Fornisce la funzionalità per riordinare manualmente le immagini caricate, permettendo all'utente di modificare l'ordine di visualizzazione.

I media, analogamente ai referenti, sono associati tramite una relazione `morph` definita nel loro modello, il che rende il componente altamente versatile e utilizzabile in qualsiasi altro modello all'interno del progetto.

Quando una foto viene aggiunta (prima di essere salvata), viene registrata automaticamente nel database come un record temporaneo, con `model_type` impostato su un valore generico (`App/Models/Attachment`) e `model_id` impostato su `NULL`.

123 id	A-Z model_type	123 model_id
3	App\Models\Attachment	[NULL]

Figura 3.15: Media prima del salvataggio.

Questo record temporaneo viene mantenuto per un massimo di sette giorni. Se l'utente non conferma il salvataggio entro tale periodo, l'immagine viene eliminata automaticamente.

Quando l'utente salva l'immagine, grazie alla relazione `morph`, i campi `model_type` e `model_id` vengono automaticamente valorizzati. Per esempio, se ad un'offerta viene aggiunta un'immagine come allegato e successivamente salvata, in database verrà registrato un record con `model_type = App\Models\Offer` e `model_id` uguale all'ID dell'offerta (in questo caso, 19).

123 id ▼	A-Z model_type ▼	123 model_id ▼
3	App\Models\Offer	19

Figura 3.16: Media dopo il salvataggio.

Per garantire il corretto funzionamento dell'intero circuito di rilevamento dei modelli, è sufficiente aggiungere il campo `protected $fillableMedia = ['attachment'];` all'interno del modello in cui si desidera gestire i media, in questo caso sia nel modello `Offer` che nei modelli `Appointment` e `Task`. Inoltre, è necessario inserire `->attachMediaFromRequest('attachment');` all'interno del controller, `OfferController`, nella sezione `store` o `update`.

3.7 Scope Filters

Oltre a filtrare per le colonne della tabella, è possibile dichiarare dei filtri personalizzati in base alle proprie esigenze. All'interno del Model delle Offerte, ad esempio, abbiamo un filtro `has_no_children` definito come segue:

```
AllowedFilter::callback('has_no_children', function (
    $query) {
    $query->whereNotIn('id', function ($subQuery) {
        $subQuery
            ->select('offer_parent_id')
            ->from('offers')
            ->whereNotNull('offer_parent_id');
    });
}),
```

Per ogni filtro personalizzato, ci deve essere la sua funzione annessa. In questo caso, filtrando per `has_no_children`, avremo all'interno del model una funzione:

```
public function scopeHasNoChildren(Builder $query)
{
    return $query->whereNotIn('id', function($subQuery) {
        $subQuery
            ->select('offer_parent_id')
            ->from('offers')
            ->whereNotNull('offer_parent_id');
    });
}
```

In questo caso, questa funzione è stata progettata per essere utilizzata nelle tabelle delle Offerte, al fine di visualizzare esclusivamente l'ultima revisione di ogni offerta, poiché sarà l'unica priva di un'offerta figlia.

3.8 Offerta Accettata

Quando viene creata un'offerta, essa può trovarsi in uno dei seguenti quattro stati: **Prevendita**, **Inviata**, **Accettata** e **Rifiutata**. Inizialmente, l'offerta è impostata come **Prevendita**, ma se vengono effettuate delle revisioni e successivamente viene raggiunto un accordo, lo stato cambierà in **Accettata**.

Al momento del salvataggio, se lo stato selezionato è **Accettata**, viene mostrata una modale di conferma.

Ogni form è avvolto dal componente **DfProForm**. Questo componente, simile ai componenti **CrudDataTable** e **FilesUploader**, dispone di alcune proprie **props**. Nel caso specifico di **DfProForm**, per fare in modo che la modale di conferma venga visualizzata solo quando lo stato è "Accettato", è sufficiente aggiungere la **prop withConfirmModal=status === "accepted"**. Ulteriori dettagli, come il titolo, il contenuto e la larghezza della modale, sono configurabili tramite la **prop confirmModalProps**, che può includere un oggetto del tipo: **title: "Titolo"**, **content: "Contenuto"**, **width: 400**, **type: "warn"**.

Una volta confermata la modifica, i campi dell'offerta diventano non modificabili e viene avviato automaticamente un progetto relativo a quell'azienda.

Questo comporta due situazioni:

- Se l'offerta accettata è di un cliente, l'offerta verrà aggiornata e il progetto verrà creato correttamente.
- Se l'offerta accettata è di un lead, quest'ultimo verrà convertito in cliente, dopodiché l'offerta verrà aggiornata e il progetto verrà creato.

Il tutto è gestito all'interno del controller, nelle funzioni `store` e `update`.

```
public function store(OfferRequest $request, Customer
    $customer)
{
    $data = $request->safe()->all();
    $parentId = isset($data['offer_parent_id']) ? $data['
offer_parent_id'] : null;

    if ($data['status'] === 'accepted' && $customer->
getAttributes()['is_customer'] === 0) {
        $customer['is_customer'] = 1;
        $customer->update([$customer]);
    }

    $code = Offer::generateCode($customer->id, $parentId)
;
    $data['code'] = $code;

    if($customer) {
        $offer = $customer->offers()->create($data);
    } else {
        return response()->json(['error' => 'Invalid
request'], 400);
    }
    $offer->attachMediaFromRequest('attachment');
    return new OfferResource($offer);
}

public function update(OfferRequest $request, ?Customer
    $customer = null, Offer $offer)
{
    $data = $request->safe()->all();
    $offer->update($data);
    $offer->attachMediaFromRequest('attachment');

    if ($data['status'] === 'accepted' && $customer->
getAttributes()['is_customer'] === 0) {
        $customer['is_customer'] = 1;
        $customer->update([$customer]);
    }
    return new OfferResource($offer);
}
```

Codice 3.3: OfferController

- **Metodo store:**

- I dati inviati dal form vengono recuperati tramite `$request->safe()->all()` e archiviati nella variabile `$data`.
- Se presente, l’ID di un’offerta genitore (`offer_parent_id`) viene estratto e salvato in `$parentId`, altrimenti viene impostato su `null`.
- Se lo stato dell’offerta è ”accepted” (accettata) e il cliente non è ancora stato contrassegnato come cliente (`is_customer` è 0), il campo `is_customer` del cliente viene aggiornato a 1.
- Viene generato un codice univoco per l’offerta utilizzando il metodo `generateCode` del modello `Offer`, che prende come parametri l’ID del cliente e l’eventuale ID dell’offerta genitore.
- Se il cliente è valido, l’offerta viene creata e associata al cliente utilizzando la relazione `offers()` e i dati elaborati.
- Se sono presenti allegati, questi vengono associati all’offerta tramite il metodo `attachMediaFromRequest('attachment')`.

Al termine, viene restituita una risposta con l’oggetto `OfferResource`, che contiene i dettagli dell’offerta appena creata.

- **Metodo update:**

- I dati inviati nel form di aggiornamento vengono estratti e archiviati nella variabile `$data`.
- L’offerta esistente viene aggiornata con i nuovi dati tramite il metodo `update()`.
- Se sono presenti allegati, questi vengono associati all’offerta tramite il metodo `attachMediaFromRequest('attachment')`.
- Se lo stato dell’offerta è ”accepted” e il cliente non è ancora segnato come cliente (`is_customer` è 0), il campo `is_customer` viene aggiornato a 1.

Anche in questo caso, il metodo restituisce un oggetto `OfferResource`, che contiene i dettagli dell’offerta aggiornata.

Capitolo 4

Possibili miglioramenti

4.1 Lavori in corso

La ristrutturazione del gestionale è attualmente in fase di sviluppo, poiché non è ancora completamente finalizzato. Nonostante le funzionalità principali siano state implementate e siano operative, ci sono ancora diverse sezioni da completare, ridefinire o modificare. Questi aggiornamenti sono essenziali per migliorare l'efficienza del sistema e per garantire che risponda pienamente alle esigenze aziendali di Dieffetech.

Il processo di revisione continua assicura che il rifacimento del gestionale si evolva in modo dinamico, migliorando costantemente la sua usabilità e la sua capacità di supportare il team. Questo approccio iterativo ci permette di affinare ogni dettaglio e di rilasciare nuove versioni con funzionalità aggiuntive, aumentando gradualmente il valore complessivo del sistema.

4.2 AI

Un potenziale miglioramento del gestionale potrebbe essere rappresentato dall'integrazione di strumenti basati sull'Intelligenza Artificiale (AI), in grado di ottimizzare i processi aziendali. In particolare, l'adozione di algoritmi di analisi predittiva consentirebbe di analizzare i dati storici relativi a clienti e lead, fornendo suggerimenti utili ai dipendenti per l'adozione di decisioni più informate. Tali algoritmi potrebbero, ad esempio, identificare con maggiore precisione i lead con maggiore probabilità di conversione o le aree aziendali che richiedono interventi strategici.

In aggiunta, l'AI potrebbe essere impiegata per l'automazione di attività ripetitive, come la generazione automatica di report periodici o la classificazione e l'archiviazione di documenti e allegati. Queste soluzioni ridurrebbero il carico di lavoro manuale dei dipendenti, permettendo loro di concentrarsi su

attività a più alto valore aggiunto. Un ulteriore sviluppo potrebbe includere l'integrazione di assistenti virtuali per supportare il personale nell'utilizzo del gestionale, semplificando e velocizzando l'accesso alle informazioni e la gestione dei processi aziendali.

4.3 Aggiornamenti e aggiunte

Uno dei punti di forza di un gestionale interno risiede nella sua capacità di adattarsi alle nuove esigenze aziendali che si presentano nel tempo. Il sistema, infatti, è progettato per essere facilmente aggiornabile e ampliabile in base alle necessità operative e strategiche dell'azienda. L'approccio modulare consente l'integrazione di nuove funzionalità, il miglioramento delle prestazioni e la risoluzione di bug, senza interrompere il flusso di lavoro.

In un contesto in continua evoluzione come quello di Dieffetech, il gestionale è soggetto a costanti aggiornamenti per riflettere cambiamenti nelle procedure aziendali, l'introduzione di nuovi strumenti tecnologici o l'integrazione di soluzioni innovative. Inoltre, aggiunte future potrebbero riguardare nuove sezioni o moduli specifici, pensati per ottimizzare ulteriormente la gestione di clienti o altri processi aziendali.

Il continuo sviluppo del gestionale rappresenta un elemento chiave per garantire che il sistema resti in linea con gli obiettivi aziendali e che supporti efficacemente i dipendenti nelle loro attività quotidiane. Questo assicura che il gestionale mantenga la sua flessibilità e scalabilità nel tempo, fornendo soluzioni sempre più aderenti alle esigenze operative dell'azienda.

Capitolo 5

Conclusione

Il rifacimento del gestionale di Dieffetech ha rappresentato un'importante sfida, ma anche un grande successo. Il lavoro svolto ha permesso di ottimizzare numerosi processi aziendali, migliorando l'efficienza operativa e facilitando la gestione delle informazioni da parte dei dipendenti. Grazie all'adozione di tecnologie moderne come React e Laravel, il sistema è diventato più robusto, scalabile e facilmente aggiornabile.

A livello professionale, questa esperienza ha costituito un'importante opportunità di crescita e sviluppo. Il progetto ha richiesto l'integrazione di diverse competenze tecniche, dalla programmazione front-end con React alla gestione del back-end con Laravel. È emersa l'importanza della collaborazione e della comunicazione tra i team di lavoro, permettendo una maggiore consapevolezza delle necessità aziendali e della loro traduzione in soluzioni informatiche. Questo percorso ha inoltre favorito lo sviluppo di competenze avanzate nella risoluzione di sfide complesse e nell'individuazione di soluzioni efficaci e scalabili.

Questo progetto non solo ha contribuito al miglioramento dei processi aziendali, ma ha anche rappresentato un passo fondamentale nella mia crescita professionale, fornendomi competenze e conoscenze che continueranno a essere fondamentali nel mio percorso futuro.

Bibliografia

- [1] Banks, A., & Porcello, E. (2019). *Learning React: Functional Web Development with React and Redux*. O'Reilly Media.
- [2] Stauffer, M. (2019). *Laravel: Up & Running: A Framework for Building Modern PHP Apps*. O'Reilly Media.
- [3] React. (n.d.). *React Documentation*. Retrieved from <https://reactjs.org/docs/getting-started.html>
- [4] Laravel. (n.d.). *Laravel Documentation*. Retrieved from <https://laravel.com/docs>
- [5] Laravel. (n.d.). *Laravel Sanctum Documentation*. Retrieved from <https://laravel.com/docs/8.x/sanctum>
- [6] Laravel. (n.d.). *Telescope Documentation*. Retrieved from <https://laravel.com/docs/8.x/telescope>
- [7] Laravel. (n.d.). *Sail Documentation*. Retrieved from <https://laravel.com/docs/8.x/sail>
- [8] Docker. (n.d.). *Docker Documentation*. Retrieved from <https://docs.docker.com>
- [9] Amazon Web Services. (n.d.). *AWS Documentation*. Retrieved from <https://docs.aws.amazon.com>
- [10] Smith, J., & Roberts, T. (2020). "Performance Optimizations in React Applications". *Journal of Web Development*. **15**(2), 45-67.

Ringraziamenti

Desidero esprimere la mia sincera gratitudine a tutte le persone che mi hanno supportato e accompagnato durante questo percorso, senza il cui sostegno questo lavoro non sarebbe stato possibile.

In primo luogo, ringrazio la mia famiglia per il loro amore incondizionato, il supporto costante e la fiducia instancabile che mi hanno dato in ogni fase del mio cammino. In particolare, desidero ringraziare mio papà, per avermi spronato ad intraprendere e proseguire con determinazione questo percorso. La loro presenza e il loro incoraggiamento sono stati la mia forza.

Un ringraziamento speciale va alla mia ragazza, per la sua pazienza, comprensione e il suo sostegno quotidiano, che mi hanno permesso di affrontare le sfide di questo progetto con serenità e determinazione. Sebbene ci siamo incontrati verso la fine del mio percorso, è stato grazie a te che sono riuscito a proseguire con fiducia e a portare a termine questo lavoro.

Un grazie di cuore va anche ai miei amici Giovanni e Cris, per la loro amicizia, i loro preziosi consigli e il supporto morale nei momenti di difficoltà. Per tutto il tempo che abbiamo condiviso insieme, tra lo sport, le avventure e i momenti di svago, che hanno reso il percorso più leggero e stimolante.

Ringrazio mia cugina Anastasija per il suo affetto e per aver sempre creduto in me, spronandomi a dare il massimo in ogni situazione.

Un sincero grazie va al mio gruppo di amici, "i topolini", con cui ho condiviso tanti momenti di allegria, crescita e supporto reciproco.

Non posso dimenticare i miei colleghi universitari Aurora Felisari, Endri Hoxha, Enrico Sardini, Leopoldo Antozzi, Luigi Botrugno, e Luigi Zheng, con i quali ho avuto il privilegio di condividere questa esperienza accademica. La loro collaborazione e amicizia sono state per me una fonte di ispirazione e di motivazione.

Desidero esprimere la mia sincera gratitudine ai miei colleghi di lavoro, i 'gelluti', per l'incredibile supporto e l'atmosfera positiva che hanno saputo creare. Ogni giorno, con passione e dedizione, hanno contribuito a rendere il

nostro ambiente lavorativo stimolante e collaborativo, sempre pronti a offrire il loro aiuto e a lavorare insieme per raggiungere obiettivi comuni.

Un ringraziamento speciale va al mio capo, Francesco Desiderio, per avermi dato l'opportunità di far parte di questo team e per aver creduto in me. Il mio sincero apprezzamento va anche al mio tutor, Federico Solfa, per il suo continuo supporto e la sua guida nei momenti di studio più impegnativi. La sua disponibilità e i suoi preziosi consigli sono stati fondamentali per il mio percorso.

A tutti voi, il mio più sincero grazie.

